UNIVERSIDAD DE LAS FUERZAS ARMADAS – ESPE



NOMBRE: Sebastián Cañizares

NRC: 36460 FECHA: 23/10/26 CARRERA: Ingeniería Electrónica y Automatización

ASIGNATURA: Fundamentos de Programación

PROBLEMA 1.1 MEDIA DE DOS VALORES

Desarrolla un programa que ingrese dos números por teclado e imprima su media.

PSEUDOCODIGO

```
Algoritmo MediaDeDosValores
    Definir num1, num2, media Como Real

    Escribir "Ingrese el primer número:"
    Leer num1
    Escribir "Ingrese el segundo número:"
    Leer num2

    media <- (num1 + num2) / 2

    Escribir "La media es: ", media
FinAlgoritmo
```

CODEBLOCKS

```
#include <stdio.h>

int main() {

    float num1, num2, media;

    printf("Ingrese el primer numero: ");

    scanf("%f", &num1);

    printf("Ingrese el segundo numero: ");

    scanf("%f", &num2);

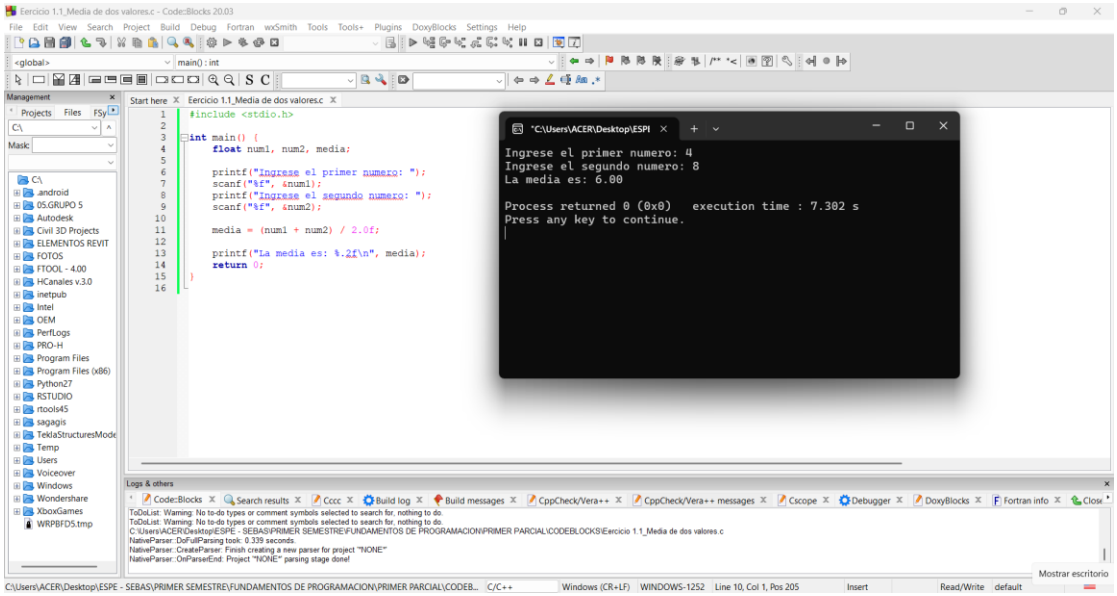

    media = (num1 + num2) / 2.0f;


    printf("La media es: %.2f\n", media);

    return 0;

}
```

CAPTURA DE EJECUCIÓN



PROBLEMA 1.2. VALOR ABSOLUTO DE X AL CUBO

Desarrolla un programa que lea un número por teclado y escriba por pantalla $|x|^3$

PSEUDOCODIGO

```
Algoritmo ValorAbsolutoAlCubo
    Definir x, abs_x, resultado Como Real

    Escribir "Ingrese el valor de x:"
    Leer x

    Si x < 0 Entonces
        abs_x <- -x
    Sino
        abs_x <- x
    FinSi

    resultado <- abs_x ^ 3

    Escribir "El resultado de |x|^3 es: ", resultado
FinAlgoritmo
```

CODEBLOCKS

```
#include <stdio.h>

#include <math.h>

int main() {

    double x, abs_x, resultado;


    printf("Ingrese el valor de x: ");

    scanf("%lf", &x);


    abs_x = (x < 0) ? -x : x;

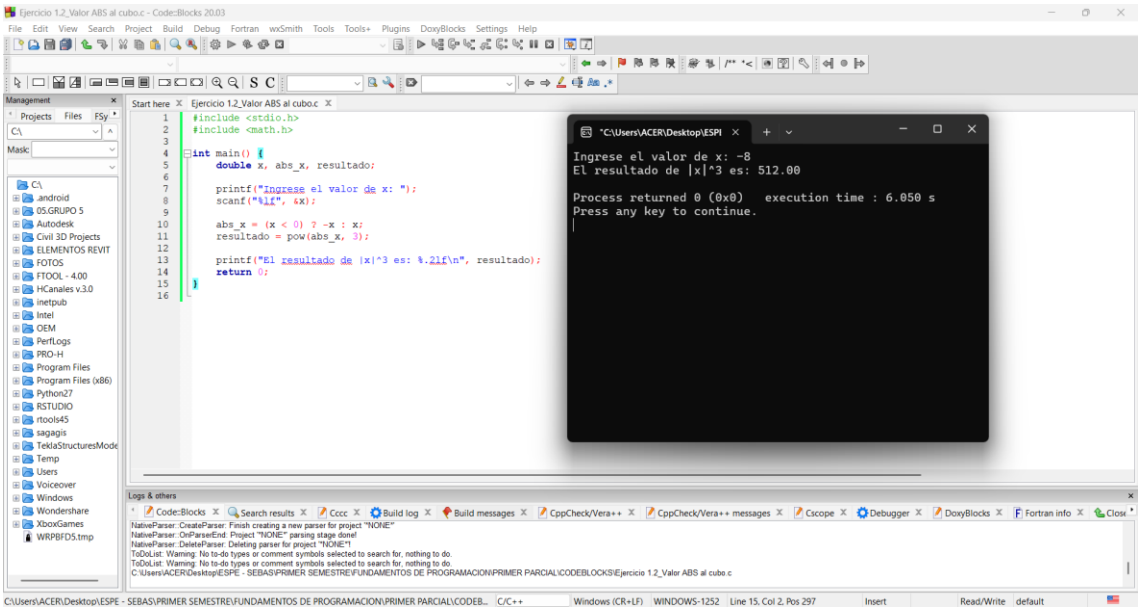
    resultado = pow(abs_x, 3);


    printf("El resultado de |x|^3 es: %.2lf\n", resultado);

    return 0;

}
```

CAPTURA EJECUCIÓN



PROBLEMA 1.3. DIVISIBLE

Desarrolla un programa que lea dos números enteros por teclado y determine si el primero es divisible por el segundo. Se mostrará por pantalla el resultado.

Pista: La solución utiliza el operador módulo (%), que devuelve el resto de la división entera entre números enteros. Si x es divisible entre y, el resto debe ser 0.

PSEUDOCODIGO

```
Algoritmo EsDivisible
    Definir num1, num2 Como Entero

    Escribir "Ingrese el primer número entero:"
    Leer num1
    Escribir "Ingrese el segundo número entero:"
    Leer num2

    Si num2 = 0 Entonces
        Escribir "Error: No se puede dividir entre cero."
    Sino
        Si num1 MOD num2 = 0 Entonces
            Escribir num1, " es divisible por ", num2
        Sino
            Escribir num1, " NO es divisible por ", num2
        FinSi
    FinSi
FinAlgoritmo
```

CODEBLOCKS

```
#include <stdio.h>

int main() {

    int num1, num2;

    printf("Ingrese el primer numero entero: ");

    scanf("%d", &num1);

    printf("Ingrese el segundo numero entero: ");

    scanf("%d", &num2);

    if (num2 == 0) {

        printf("Error: No se puede dividir entre cero.\n");

    } else if (num1 % num2 == 0) {

        printf("%d es divisible por %d\n", num1, num2);

    } else {

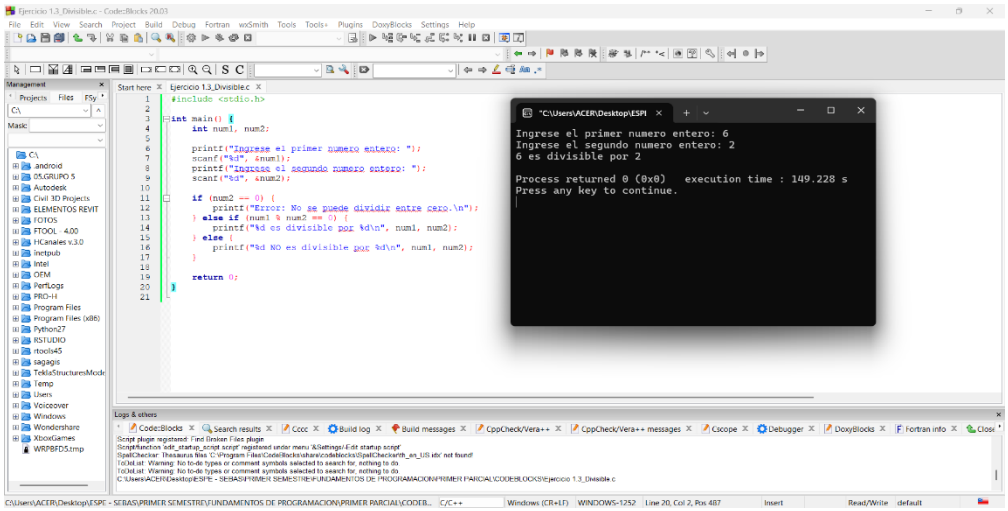
        printf("%d NO es divisible por %d\n", num1, num2);

    }

    return 0;

}
```

CAPTURA EJECUCIÓN



PROBLEMA 1.4. INTERVALO

Desarrolla un programa que lea un número por teclado y determine si pertenece al intervalo [0, 10], indicando el resultado por pantalla.

Pista: El número introducido debe cumplir $x \geq 0$ y $x \leq 10$ (ambas condiciones a la vez) para pertenecer al intervalo. Para establecer estas dos condiciones se utiliza un operador lógico: el operador Y en el diagrama de flujo y && en C. Evita la tentación de escribir $0 \leq x \leq 10$ como una sola condición, porque en C no se evalúa como esperas.

PSEUDOCODIGO

```
Algoritmo PerteneceAlIntervalo
    Definir x Como Real

    Escribir "Ingrese un número:"
    Leer x

    Si (x >= 0) Y (x <= 10) Entonces
        Escribir "El número ", x, " pertenece al intervalo [0, 10]"
    Sino
        Escribir "El número ", x, " NO pertenece al intervalo [0, 10]"
    FinSi
FinAlgoritmo
```

CODEBLOCKS

```
#include <stdio.h>

int main() {

    float x;

    printf("Ingrese un numero: ");

    scanf("%f", &x);


    if (x >= 0 && x <= 10) {

        printf("El numero %.2f pertenece al intervalo [0, 10]\n", x);

    } else {

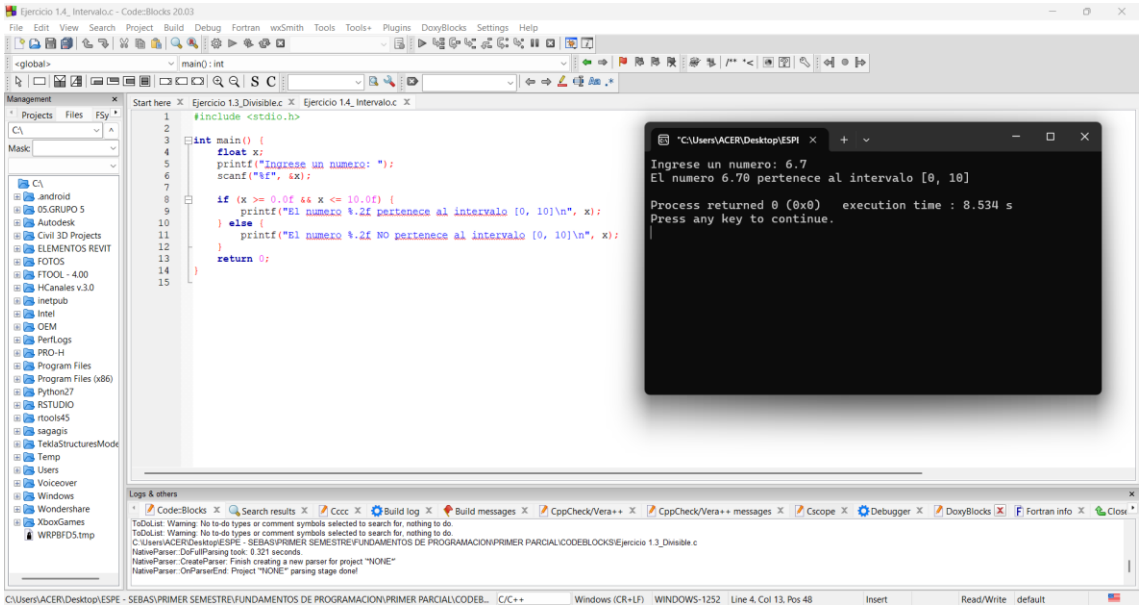
        printf("El numero %.2f NO pertenece al intervalo [0, 10]\n", x);

    }


    return 0;

}
```

CAPTURA DE EJECUCIÓN



PROBLEMA 1.5. CONVERSIÓN DE UNIDADES DE TIEMPO

Desarrolla un programa que lea por teclado un valor entero c, interpretándolo como el número de segundos que dura un evento, y calcule y muestre por pantalla cuántos días, horas, minutos y segundos representa.

Para calcular los días, horas, minutos y segundos se utilizan la división entera y el módulo (resto de una división, operador %). Sea c el número de segundos a convertir. Como un día tiene 24 horas, una hora tiene 60 minutos y un minuto tiene 60 segundos, el cociente entero c / (24 · 60 · 60) es el número de días completos contenidos en c. Asimismo, c % (24 · 60 · 60) es el número de segundos restantes tras descontar esos días. Se procede de igual manera para obtener las horas (1 hora = 60 · 60 segundos) y los minutos (1 minuto = 60 segundos).

Ejemplo: supón que quieres convertir c = 10 000 segundos. Como 10 000 / (24 · 60 · 60) = 0, hay 0 días, porque 10 000 segundos es menos que un día. Análogamente, 10 000 / (60 · 60) = 2 horas. Si a 10 000 segundos les quitas esas 2 horas, quedan 10 000 % (60 · 60) = 2 800 segundos, que son algo más de 46 minutos, cantidad que se obtiene con 2 800 / 60 = 46 minutos. Finalmente, 2 800 % 60 = 40 segundos. Por lo tanto, 10 000 segundos equivalen a 0 días, 2 horas, 46 minutos y 40 segundos.

PSEUDOCODIGO

```
Algoritmo ConversionTiempo

    Definir c, dias, horas, minutos, segundos, resto Como Entero

    Escribir "Ingrese la cantidad de segundos:"

    Leer c

    dias <- c / 86400

    resto <- c MOD 86400

    horas <- resto / 3600

    resto <- resto MOD 3600

    minutos <- resto / 60

    segundos <- resto MOD 60
```

Escribir c, " segundos equivalen a: ", dias, " días, ", horas, " horas, ", minutos, " minutos y ", segundos, " segundos."

FinAlgoritmo

CODEBLOCKS

```
#include <stdio.h>

int main() {
    int c, dias, horas, minutos, segundos, resto;

    printf("Ingrese la cantidad de segundos: ");
    scanf("%d", &c);

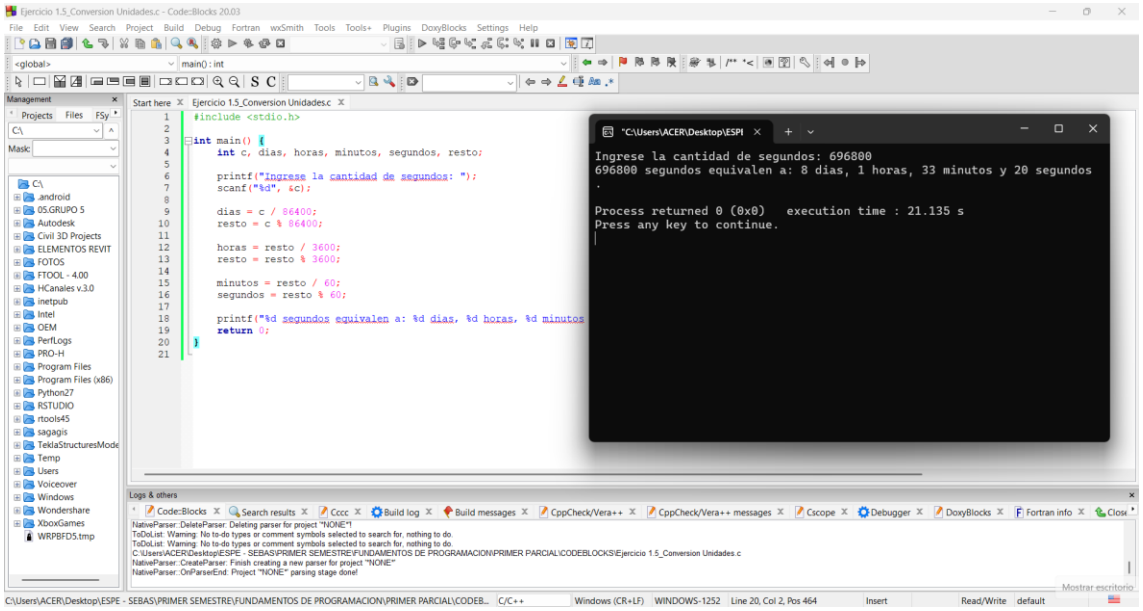
    dias = c / 86400;
    resto = c % 86400;

    horas = resto / 3600;
    resto = resto % 3600;

    minutos = resto / 60;
    segundos = resto % 60;

    printf("%d segundos equivalen a: %d dias, %d horas, %d minutos y %d segundos.\n", c, dias, horas, minutos, segundos);
    return 0;
}
```

CAPTURA DE EJECUCIÓN



PROBLEMA 1.6. MÁXIMO, MÍNIMO Y MEDIA DE ALTURAS

Desarrolla un programa que lea por teclado un número indeterminado de alturas y deje de leer cuando se introduzca una negativa. El programa debe calcular e imprimir la media, el máximo y el mínimo de las alturas introducidas.

PSEUDOCODIGO

Algoritmo MaxMinMediaAlturas

Definir h, suma, max, min, media Como Real

Definir contador Como Entero

suma <- 0

contador <- 0

Escribir "Ingrese una altura (o número negativo para salir):"

Leer h

```
Mientras h >= 0 Hacer
    suma <- suma + h
    contador <- contador + 1

    Si contador = 1 Entonces
        max <- h
        min <- h
    Sino
        Si h > max Entonces
            max <- h
        FinSi
        Si h < min Entonces
            min <- h
        FinSi
    FinSi

    Escribir "Ingrese otra altura (o número negativo para salir):"
    Leer h
FinMientras

Si contador > 0 Entonces
    media <- suma / contador
    Escribir "Media de alturas: ", media
    Escribir "Altura máxima: ", max
```

```
        Escribir "Altura mínima: ", min
    Sino
        Escribir "No se ingresaron alturas válidas."
    FinSi
FinAlgoritmo
```

CODEBLOCKS

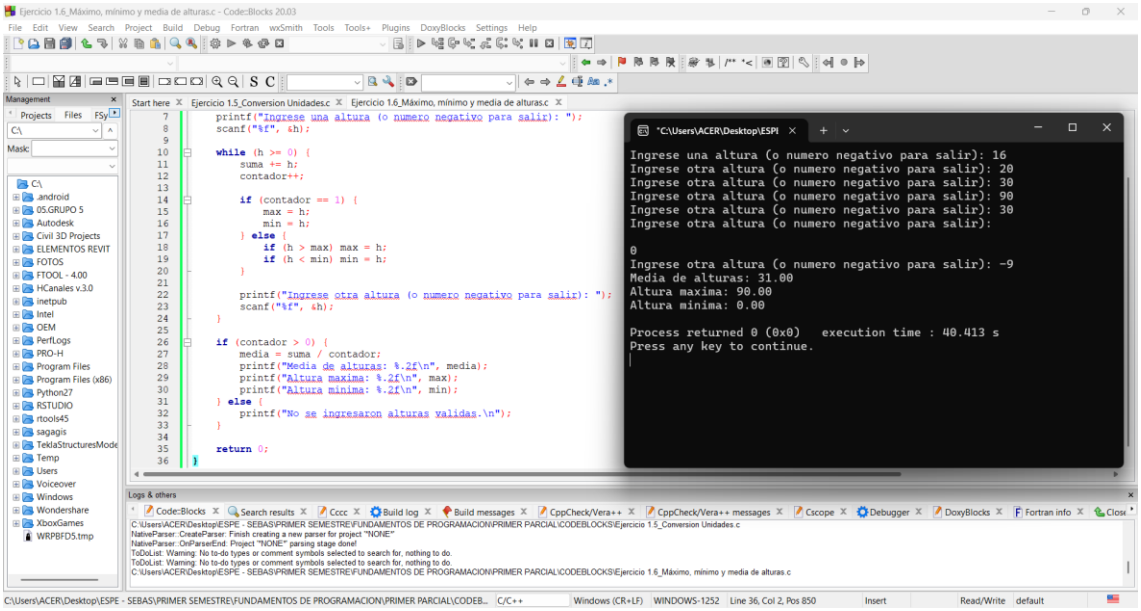
```
#include <stdio.h>
int main() {
    float h, suma = 0, max = 0, min = 0, media;
    int contador = 0;
    printf("Ingrese una altura (o numero negativo para salir): ");
    scanf("%f", &h)
    while (h >= 0) {
        suma += h;
        contador++;

        if (contador == 1) {
            max = h;
            min = h;
        } else {
            if (h > max) max = h;
            if (h < min) min = h;
        }
```

```
        printf("Ingrese otra altura (o numero negativo para salir): ");
        scanf("%f", &h);
    }

    if (contador > 0) {
        media = suma / contador;
        printf("Media de alturas: %.2f\n", media);
        printf("Altura maxima: %.2f\n", max);
        printf("Altura minima: %.2f\n", min);
    } else {
        printf("No se ingresaron alturas validas.\n");
    }
    return 0;
}
```

CAPTURA EJECUCIÓN



PROBLEMA 1.7. RAÍZ CUADRADA ENTERA

Desarrolla un programa que calcule la raíz cuadrada entera por defecto de un número N positivo dado, que se define como el mayor número entero $R > 0$ tal que $R^2 \leq N$.

Pista: Dada una variable entera i = 1, itera incrementando su valor de uno en uno mientras $i^2 \leq N$. La aproximación por defecto de la raíz es $R = i - 1$.

PSEUDOCODIGO

```
Algoritmo RaizCuadradaEntera

   Definir N, i, R Como Entero


   Escribir "Ingrese un número entero positivo N:"

   Leer N


   i <- 1

   Mientras (i * i) <= N Hacer

      i <- i + 1

   FinMientras


   R <- i - 1


   Escribir "La raíz cuadrada entera por defecto de ", N, " es: ", R

FinAlgoritmo
```

CODEBLOCKS

```
#include <stdio.h>


int main() {
```

```
    int N, i = 1, R;

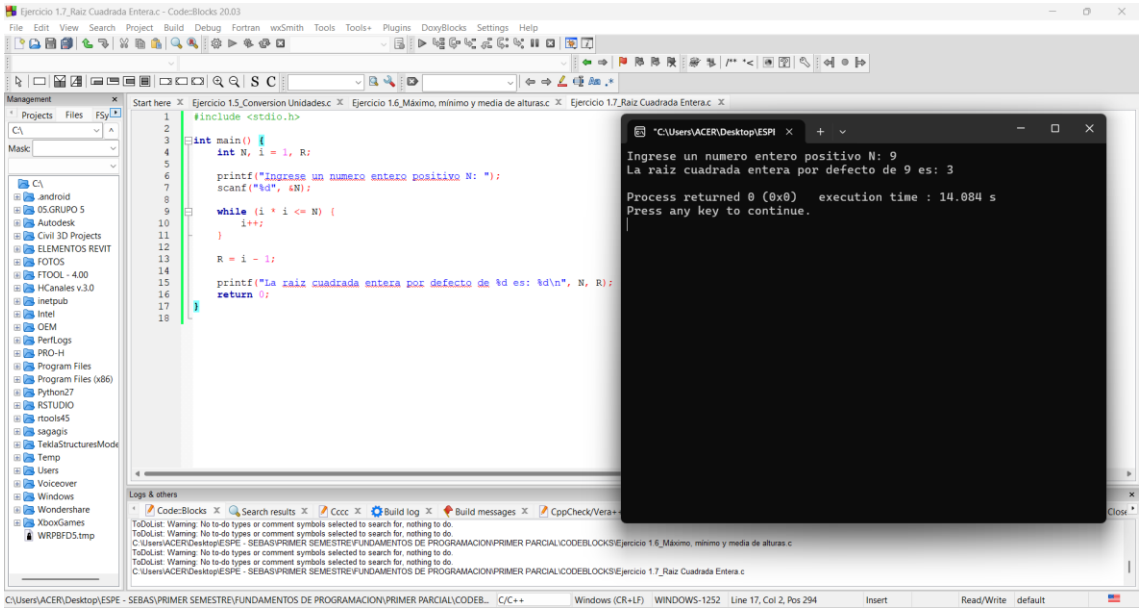
    printf("Ingrese un numero entero positivo N: ");
    scanf("%d", &N);

    while (i * i <= N) {
        i++;
    }

    R = i - 1;

    printf("La raiz cuadrada entera por defecto de %d es: %d\n", N, R);
    return 0;
}
```

CAPTURA EJECUCIÓN



PROBLEMA 1.8. NÚMERO PRIMO

Desarrolla un programa que ingrese por teclado un número entero mayor que 1 y determine si es primo o no.

PSEUDOCODIGO

```
Algoritmo NumeroPrimo
    Definir num, i Como Entero
    Definir es_primo Como Logico

    Escribir "Ingrese un número entero mayor que 1:"
    Leer num

    es_primo <- Verdadero
    i <- 2

    Mientras (i * i <= num) Y es_primo Hacer
        Si num MOD i = 0 Entonces
            es_primo <- Falso
        Sino
            i <- i + 1
        FinSi
    FinMientras

    Si es_primo Y num > 1 Entonces
        Escribir num, " es un número primo."
    Sino
```

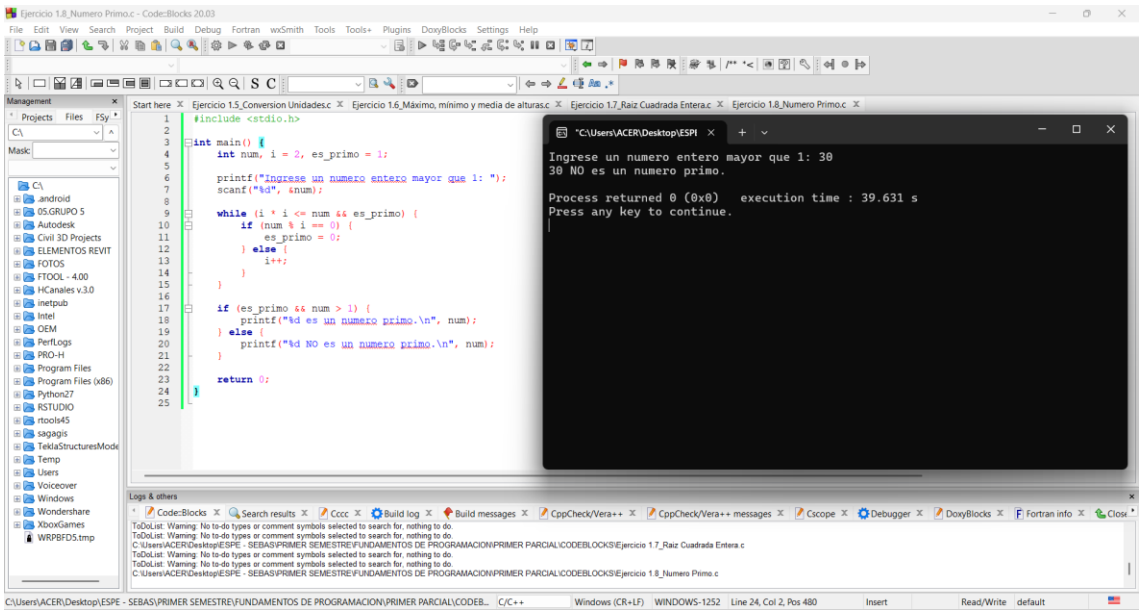
```
        Escribir num, " NO es un número primo."
    FinSi
FinAlgoritmo
```

CODEBLOCKS

```
#include <stdio.h>
int main() {
    int num, i = 2, es_primo = 1;
    printf("Ingrese un numero entero mayor que 1: ");
    scanf("%d", &num);
    while (i * i <= num && es_primo) {
        if (num % i == 0) {
            es_primo = 0;
        } else {
            i++;
        }
    }
    if (es_primo && num > 1) {
        printf("%d es un numero primo.\n", num);
    } else {
        printf("%d NO es un numero primo.\n", num);
    }

    return 0;
}
```

CAPTURA DE EJECUCIÓN



PROBLEMA 1.9. NÚMEROS PRIMOS COMPRENDIDOS ENTRE DOS NÚMEROS DADOS

Desarrolla un programa que muestre por pantalla todos los números primos comprendidos entre dos números enteros introducidos por teclado.

Desafíos de algoritmia Estos once ejercicios suben poco a poco de dificultad. En cada uno encontrarás una pregunta “Para pensar”: respóndela por escrito, porque la rúbrica final valora tu capacidad de justificar y evaluar tus decisiones.

PSEUDOCODIGO

Algoritmo PrimosEnRango

Definir a, b, inicio, fin, num, i Como Entero

Definir es_primo Como Logico

Escribir "Ingrese el primer número:"

Leer a

Escribir "Ingrese el segundo número:"

Leer b

```
Si a <= b Entonces
   inicio <- a
   fin <- b
Sino
   inicio <- b
   fin <- a
FinSi

Escribir "Números primos en el rango [", inicio, ", ", fin, "]:"

Para num <- inicio Hasta fin Hacer
   Si num > 1 Entonces
      es_primo <- Verdadero
      i <- 2
      Mientras (i * i <= num) Y es_primo Hacer
         Si num MOD i = 0 Entonces
            es_primo <- Falso
         Sino
            i <- i + 1
         FinSi
      FinMientras

      Si es_primo Entonces
```

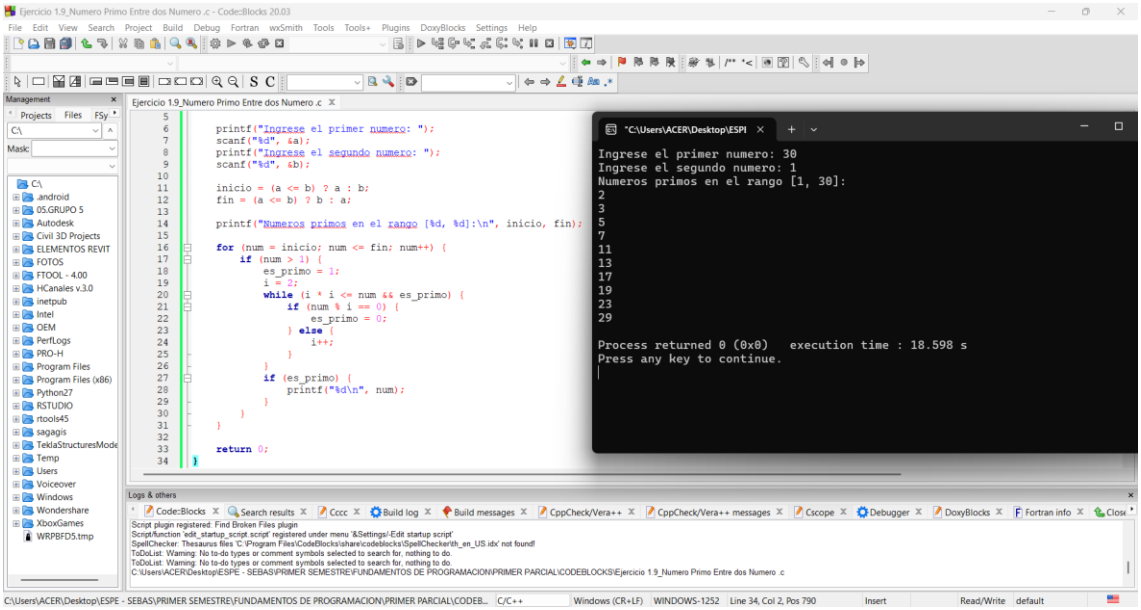
```
            Escribir num
        FinSi
    FinSi
  FinPara
FinAlgoritmo
```

CODEBLOCKS

```
#include <stdio.h>
int main() {
    int a, b, inicio, fin, num, i, es_primo;
    printf("Ingrese el primer numero: ");
    scanf("%d", &a);
    printf("Ingrese el segundo numero: ");
    scanf("%d", &b);
    inicio = (a <= b) ? a : b;
    fin = (a <= b) ? b : a;
    printf("Numeros primos en el rango [%d, %d]:\n", inicio, fin);
    for (num = inicio; num <= fin; num++) {
        if (num > 1) {
            es_primo = 1;
            i = 2;
            while (i * i <= num && es_primo) {
                if (num % i == 0) {
                    es_primo = 0;
                } else {
```

```
            i++;
        }
    }
    if (es_primo) {
        printf("%d\n", num);
    }
  }
 }
 return 0;
}
```

CAPTURA DE EJECUCIÓN



PROBLEMA 1.10. FACTORIAL

¡Desarrolla un programa que lea un número entero n ≥ 0 y calcule n! = 1 · 2 · … · n, sabiendo que 0! = 1. Ejemplo: n = 5 → 120.

Pista: Usa un acumulador que inicie en 1 y se multiplique en cada vuelta del ciclo.

Para pensar: ¿Qué ocurre si el usuario ingresa un número negativo? ¿Y con n = 20 en una variable de tipo int?

PSEUDOCODIGO

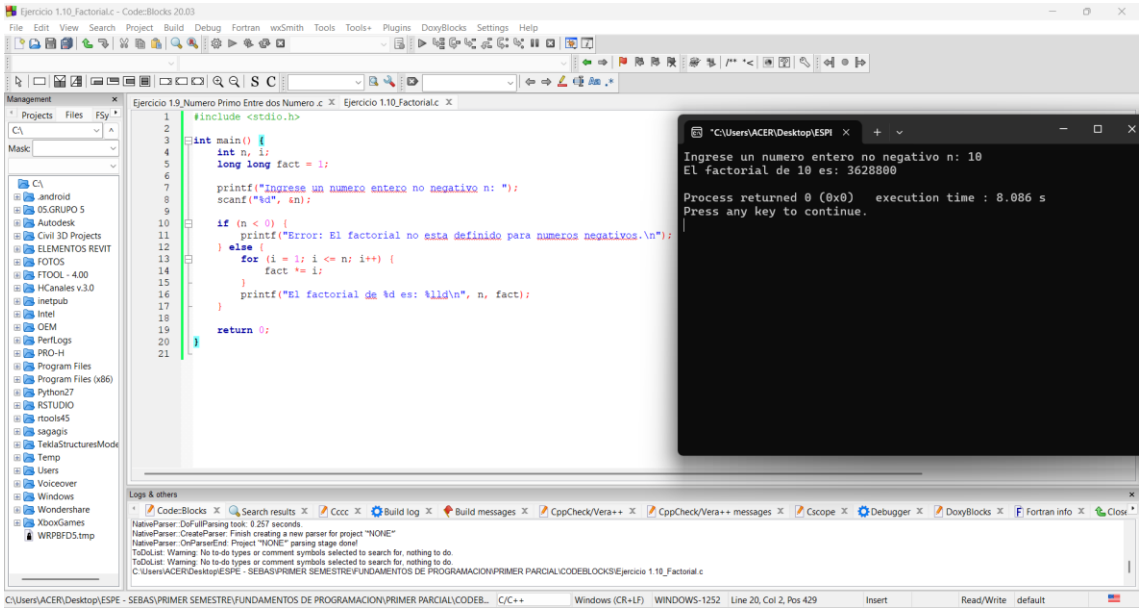
```
Algoritmo CalcularFactorial
    Definir n, i Como Entero
    Definir fact Como Entero
    Escribir "Ingrese un número entero no negativo n:"
    Leer n
    Si n < 0 Entonces
        Escribir "Error: El factorial no está definido para números negativos."
    Sino
        fact <- 1
        Para i <- 1 Hasta n Hacer
            fact <- fact * i
        FinPara
        Escribir "El factorial de ", n, " es: ", fact
    FinSi
FinAlgoritmo
```

CODEBLOCKS

```
#include <stdio.h>
int main() {
    int n, i;
    long long fact = 1;
    printf("Ingrese un numero entero no negativo n: ");
    scanf("%d", &n);
```

```
    if (n < 0) {
        printf("Error: El factorial no esta definido para numeros negativos.\n");
    } else {
        for (i = 1; i <= n; i++) {
            fact *= i;
        }
        printf("El factorial de %d es: %lld\n", n, fact);
    }
    return 0;
}
```

CAPTURA DE EJECUCIÓN



PROBLEMA 1.11 SERIE DE FIBONACCI

Desarrolla un programa que lea un entero n ≥ 1 y muestre los primeros n términos de la serie de Fibonacci (0, 1, 1, 2, 3, 5, 8, …), donde cada término es la suma de los dos anteriores.

Ejemplo: n = 7 → 0 1 1 2 3 5 8. Pista: Necesitas tres variables: el término anterior, el actual y el siguiente.

Para pensar: ¿Cómo cambia tu algoritmo si solo debes mostrar el término número n y no toda la serie?

PSEUDOCODIGO

```
Algoritmo SerieFibonacci
    Definir n, a, b, siguiente, i Como Entero

    Escribir "Ingrese la cantidad de términos n (n >= 1):"
    Leer n

    Si n < 1 Entonces
        Escribir "Error: n debe ser mayor o igual a 1."
    Sino
        a <- 0
        b <- 1

        Escribir "Los primeros ", n, " términos de Fibonacci son:"
        Para i <- 1 Hasta n Hacer
            Escribir a
            siguiente <- a + b
            a <- b
            b <- siguiente
        FinPara
    FinSi
```

FinAlgoritmo

CODEBLOCKS

```
#include <stdio.h>

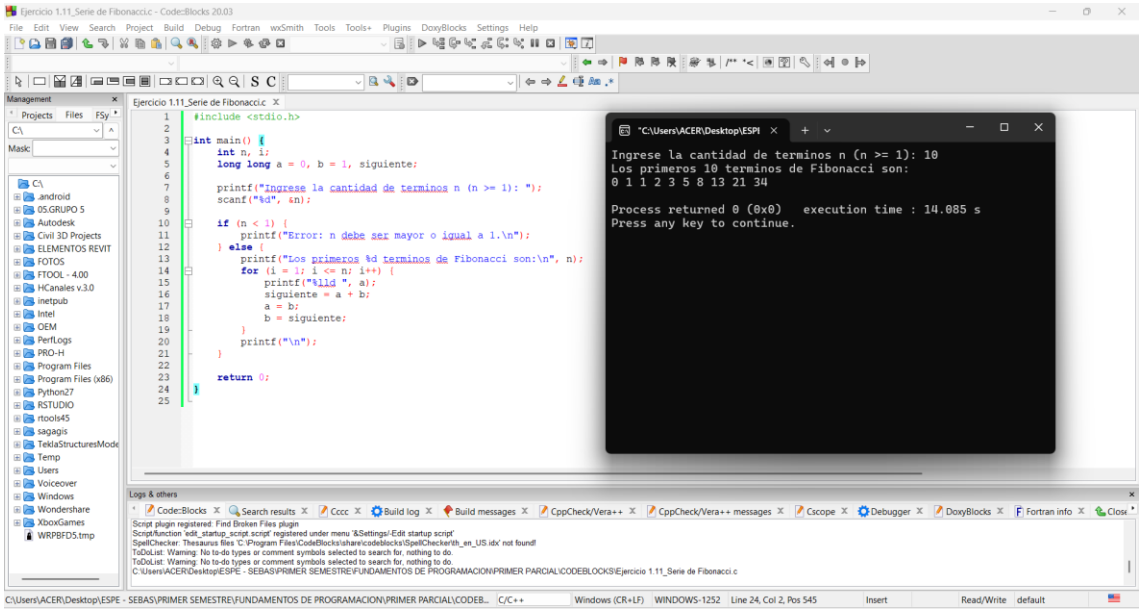

int main() {
    int n, i;
    long long a = 0, b = 1, siguiente;


    printf("Ingrese la cantidad de terminos n (n >= 1): ");
    scanf("%d", &n);


    if (n < 1) {
        printf("Error: n debe ser mayor o igual a 1.\n");
    } else {
        printf("Los primeros %d terminos de Fibonacci son:\n", n);
        for (i = 1; i <= n; i++) {
            printf("%lld ", a);
            siguiente = a + b;
            a = b;
            b = siguiente;
        }
        printf("\n");
    }
```

```
    return 0;
}
```

CAPTURA DE EJECUCIÓN



PROBLEMA 1.12 SUMA DE DÍGITOS

Desarrolla un programa que lea un entero positivo y calcule la suma de sus dígitos. Ejemplo: 4 825 → 4 + 8 + 2 + 5 = 19.

Pista: El resto de dividir entre 10 (n % 10) te da el último dígito; la división entera entre 10 lo elimina.

Para pensar: ¿Cuántas veces se repite tu ciclo para un número de k dígitos? Justifica tu respuesta.

PSEUDOCODIGO

```
Algoritmo SumaDeDigitos

    Definir num, suma, digito Como Entero


    Escribir "Ingrese un número entero positivo:"

    Leer num
```

```
    suma <- 0

    Mientras num > 0 Hacer
        digito <- num MOD 10
        suma <- suma + digito
        num <- num / 10
    FinMientras

    Escribir "La suma de los dígitos es: ", suma
FinAlgoritmo
```

CODEBLOCKS

```
#include <stdio.h>

int main() {
    int num, suma = 0, digito;

    printf("Ingrese un numero entero positivo: ");
    scanf("%d", &num);

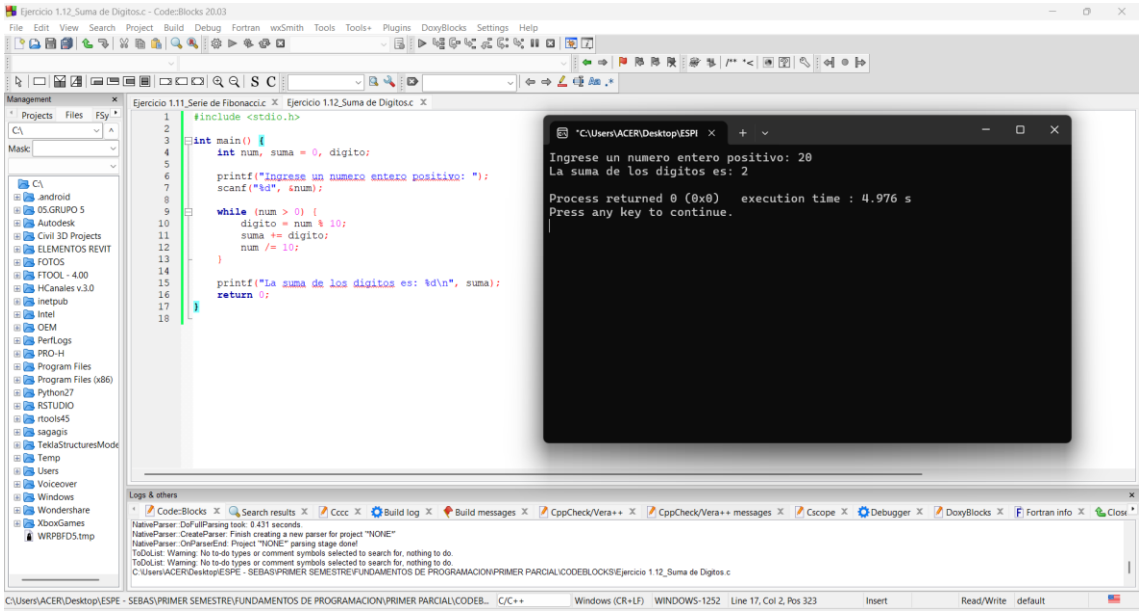
    while (num > 0) {
        digito = num % 10;
        suma += digito;
        num /= 10;
    }
```

```
    printf("La suma de los digitos es: %d\n", suma);

    return 0;

}
```

CAPTURA DE EJECUCIÓN



PROBLEMA 1.13 NÚMERO CAPICÚA

Desarrolla un programa que lea un entero positivo y determine si es capicúa, es decir, si se lee igual de izquierda a derecha que de derecha a izquierda. Ejemplos: 12321 es capicúa; 1231 no lo es.

Pista: Construye el número invertido con el mismo recurso del ejercicio 1.12 y compáralo con el original.

Para pensar: ¿Qué casos de prueba elegirías para estar seguro/a de que tu solución es correcta (por ejemplo, números de un solo dígito o que terminan en 0)?

PSEUDOCODIGO

```
Algoritmo NumeroCapicua

    Definir num, temp, digito, invertido Como Entero
```

```
    Escribir "Ingrese un número entero positivo:"
    Leer num

    temp <- num
    invertido <- 0

    Mientras temp > 0 Hacer
        digito <- temp MOD 10
        invertido <- (invertido * 10) + digito
        temp <- temp / 10
    FinMientras

    Si num = invertido Entonces
        Escribir "El número ", num, " es capicúa."
    Sino
        Escribir "El número ", num, " NO es capicúa."
    FinSi
FinAlgoritmo
```

CODEBLOCKS

```
#include <stdio.h>

int main() {
    int num, temp, digito, invertido = 0;
```

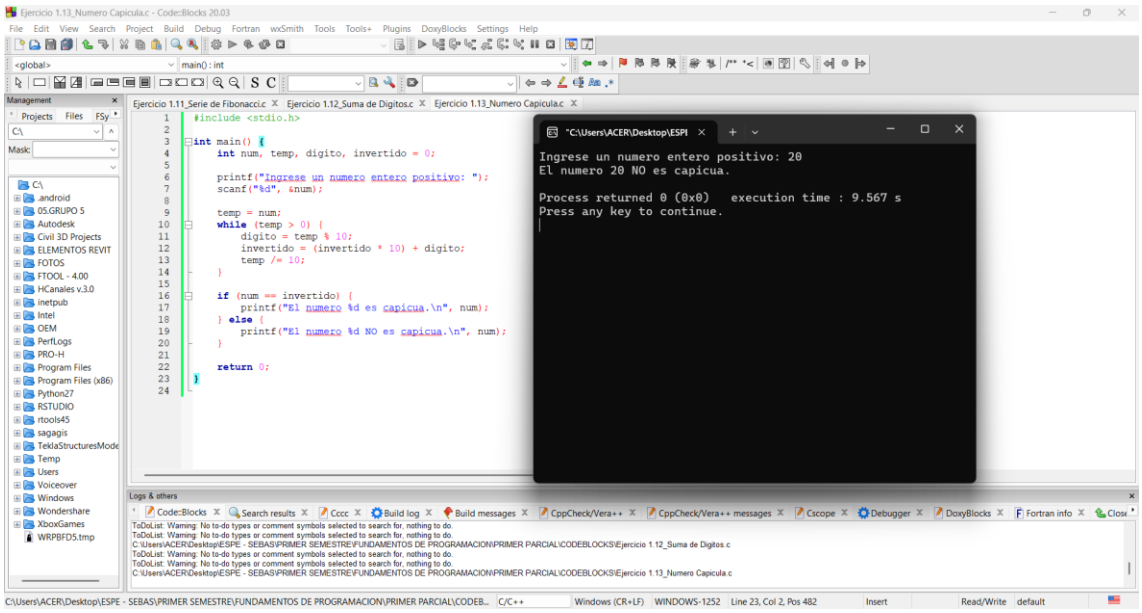
```
    printf("Ingrese un numero entero positivo: ");
    scanf("%d", &num);

    temp = num;
    while (temp > 0) {
        digito = temp % 10;
        invertido = (invertido * 10) + digito;
        temp /= 10;
    }

    if (num == invertido) {
        printf("El numero %d es capicua.\n", num);
    } else {
        printf("El numero %d NO es capicua.\n", num);
    }

    return 0;
}
```

CAPTURA DE EJECUCIÓN



PROBLEMA 1.14 MÁXIMO COMÚN DIVISOR (ALGORITMO DE EUCLIDES)

Desarrolla un programa que lea dos enteros positivos a y b y calcule su máximo común divisor con el algoritmo de Euclides: mientras b ≠ 0, reemplaza (a, b) por (b, a % b). Al terminar, a es el MCD. Ejemplo: MCD(48, 18) = 6. Pista: Haz la prueba de escritorio con (48, 18): los pares sucesivos son (48, 18), (18, 12), (12, 6) y (6, 0).

Para pensar: Compara este método con probar todos los divisores desde 1 hasta el menor de los dos números. ¿Cuál hace menos operaciones y por qué?

PSEUDOCODIGO

Algoritmo MCDEuclides

Definir a, b, resto Como Entero

Escribir "Ingrese el primer número positivo a:"

Leer a

Escribir "Ingrese el segundo número positivo b:"

Leer b

Mientras b <> 0 Hacer

```
        resto <- a MOD b
        a <- b
        b <- resto
    FinMientras

    Escribir "El Máximo Común Divisor (MCD) es: ", a
FinAlgoritmo
```

CODEBLOCKS

```
#include <stdio.h>

int main() {
    int a, b, resto;

    printf("Ingrese el primer numero positivo a: ");
    scanf("%d", &a);
    printf("Ingrese el segundo numero positivo b: ");
    scanf("%d", &b);

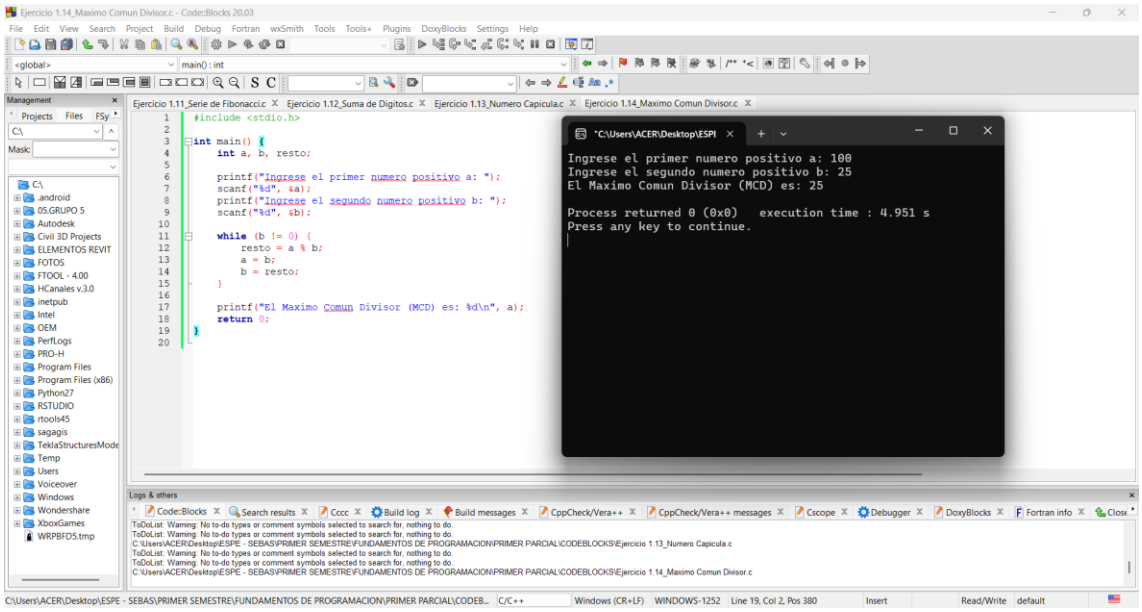
    while (b != 0) {
        resto = a % b;
        a = b;
        b = resto;
    }
```

```
    printf("El Maximo Comun Divisor (MCD) es: %d\n", a);

    return 0;

}
```

CAPTURA DE EJECUCIÓN



PROBLEMA 1.15 AÑO BISIESTO

Desarrolla un programa que lea un año y determine si es bisiesto. Un año es bisiesto si es divisible entre 4, excepto los años divisibles entre 100, que solo lo son si también son divisibles entre 400. Ejemplos: 2024 sí; 1900 no; 2000 sí.

Pista: Combina los operadores &&, || y % en una sola condición, o utiliza condiciones anidadas. Para pensar: ¿Es equivalente escribir las condiciones en otro orden? Prueba con 1900 y 2000 para comprobarlo.

PSEUDOCODIGO

```
Algoritmo AnioBisiesto

    Definir anio Como Entero


    Escribir "Ingrese un año:"

    Leer anio
```

```
    Si ((anio MOD 4 = 0) Y (anio MOD 100 <> 0)) O (anio MOD 400 = 0) Entonces
        Escribir "El año ", anio, " es bisiesto."
    Sino
        Escribir "El año ", anio, " NO es bisiesto."
    FinSi
FinAlgoritmo
```

CODEBLOCKS

```
#include <stdio.h>

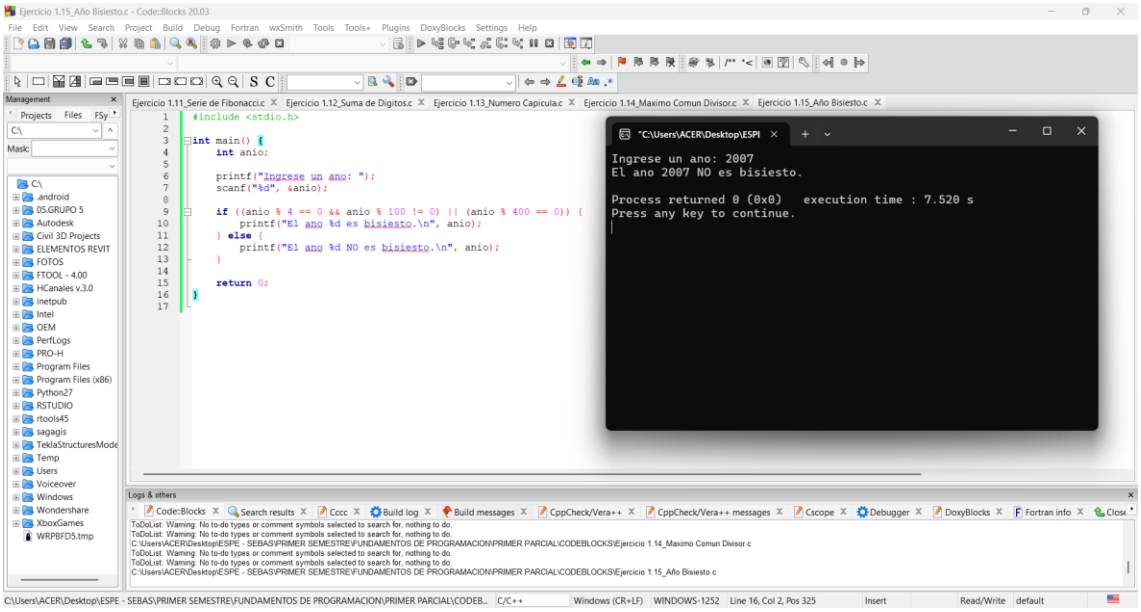
int main() {
    int anio;

    printf("Ingrese un ano: ");
    scanf("%d", &anio);

    if ((anio % 4 == 0 && anio % 100 != 0) || (anio % 400 == 0)) {
        printf("El ano %d es bisiesto.\n", anio);
    } else {
        printf("El ano %d NO es bisiesto.\n", anio);
    }

    return 0;
}
```

CAPTURA DE EJECUCIÓN



PROBLEMA 1.16 NÚMERO PERFECTO

Desarrolla un programa que lea un entero positivo y determine si es perfecto, es decir, si es igual a la suma de sus divisores propios (los menores que él). Ejemplo: 28 = 1 + 2 + 4 + 7 + 14, por lo tanto es perfecto.

Pista: Recorre los posibles divisores desde 1 hasta n − 1 y acumula los que cumplan n % d == 0. Para pensar: ¿Hasta qué valor es realmente necesario recorrer los divisores? Explica cómo reducirías el número de vueltas.

PSEUDOCODIGO

Algoritmo NumeroPerfecto

Definir n, d, suma_div Como Entero

Escribir "Ingrese un número entero positivo:"

Leer n

suma_div <- 0

```
    Para d <- 1 Hasta n - 1 Hacer
        Si n MOD d = 0 Entonces
            suma_div <- suma_div + d
        FinSi
    FinPara

    Si suma_div = n Entonces
        Escribir "El número ", n, " es perfecto."
    Sino
        Escribir "El número ", n, " NO es perfecto."
    FinSi
FinAlgoritmo
```

CODEBLOCKS

```
#include <stdio.h>

int main() {
    int n, d, suma_div = 0;

    printf("Ingrese un numero entero positivo: ");
    scanf("%d", &n);

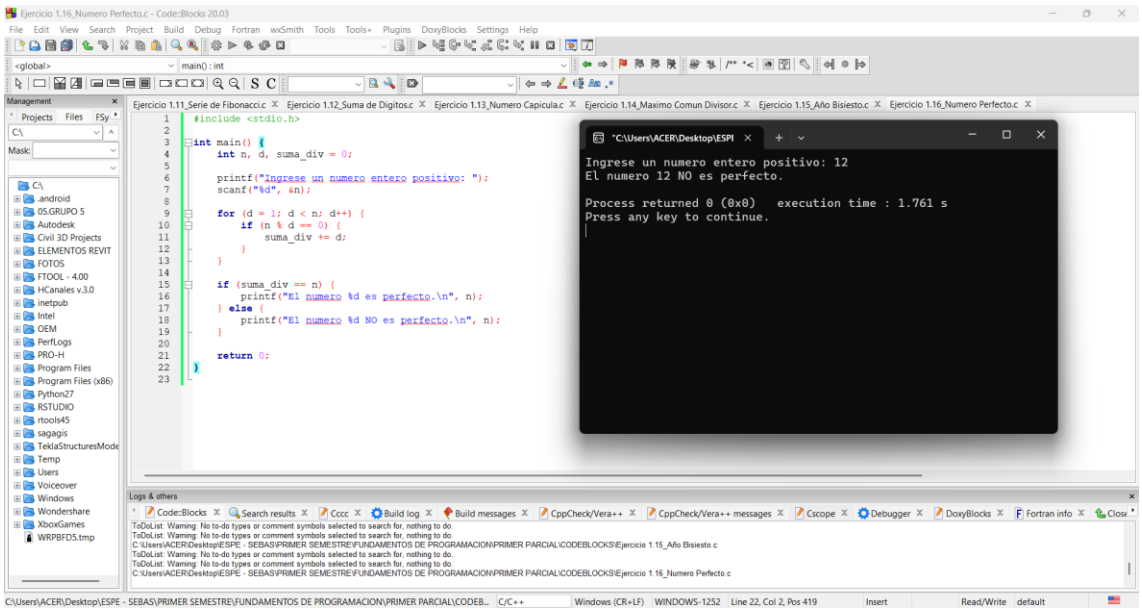
    for (d = 1; d < n; d++) {
        if (n % d == 0) {
            suma_div += d;
```

```
    }
}

if (suma_div == n) {
    printf("El numero %d es perfecto.\n", n);
} else {
    printf("El numero %d NO es perfecto.\n", n);
}

return 0;
}
```

CAPTURA DE EJECUCIÓN



PROBLEMA 1.17 CONVERSIÓN DE DECIMAL A BINARIO

Desarrolla un programa que lea un entero positivo en base 10 y muestre su representación en binario. Ejemplo: 13 → 1101.

Pista: Divide entre 2 sucesivamente y guarda los restos; el resultado se lee de abajo hacia arriba.

Para pensar: Los restos se obtienen en orden inverso al que se leen. ¿Cómo resolverías ese detalle sin usar arreglos?

PSEUDOCODIGO

```
Algoritmo DecimalABinario
    Definir n, temp, resto, binario, posicion Como Entero

    Escribir "Ingrese un número entero positivo:"
    Leer n

    temp <- n
    binario <- 0
    posicion <- 1

    Mientras temp > 0 Hacer
        resto <- temp MOD 2
        binario <- binario + (resto * posicion)
        posicion <- posicion * 10
        temp <- temp / 2
    FinMientras

    Escribir "El número ", n, " en binario es: ", binario
FinAlgoritmo
```

CODEBLOCKS

```
#include <stdio.h>
```

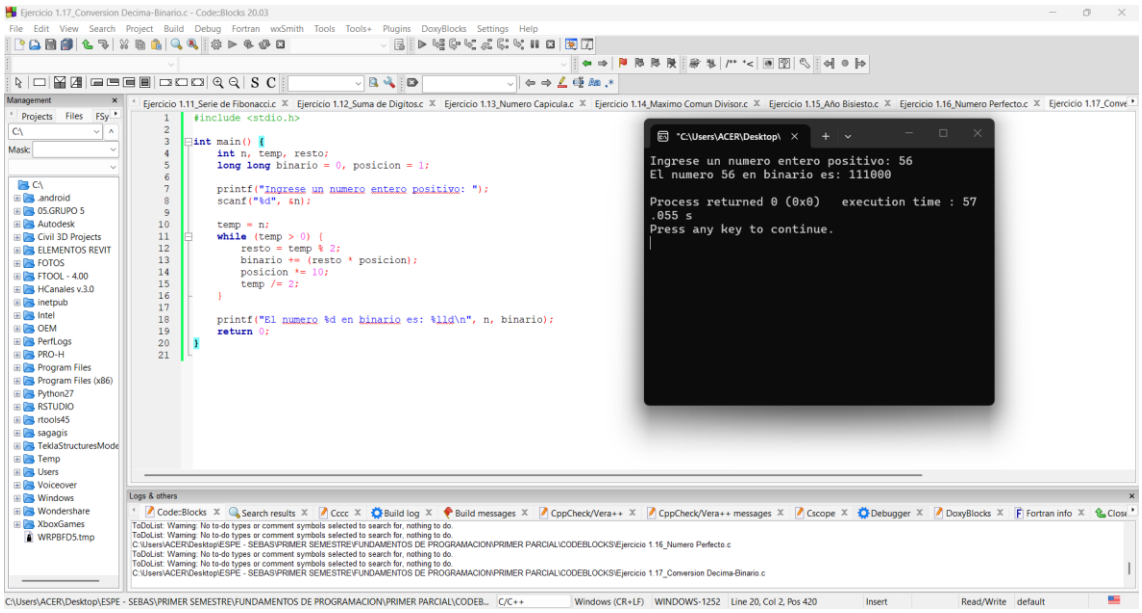
```
int main() {
    int n, temp, resto;
    long long binario = 0, posicion = 1;

    printf("Ingrese un numero entero positivo: ");
    scanf("%d", &n);

    temp = n;
    while (temp > 0) {
        resto = temp % 2;
        binario += (resto * posicion);
        posicion *= 10;
        temp /= 2;
    }

    printf("El numero %d en binario es: %lld\n", n, binario);
    return 0;
}
```

CAPTURA DE EJECUCIÓN



PROBLEMA 1.18 TRIÁNGULO DE ASTERISCOS

Desarrolla un programa que lea un entero n ≥ 1 y dibuje por pantalla un triángulo rectángulo de n filas, donde la fila i tiene i asteriscos. Ejemplo con n = 4: * ** *** **** Pista: Usa un ciclo dentro de otro: el externo controla las filas y el interno, los asteriscos de cada fila.

Para pensar: Si ahora quisieras un triángulo centrado (una pirámide), ¿qué parte de tu algoritmo cambiaría y qué parte se mantendría?

PSEUDOCODIGO

Algoritmo TrianguloAsteriscos

Definir n, i, j Como Entero

Escribir "Ingrese un número entero n (n >= 1):"

Leer n

Si n < 1 Entonces

Escribir "Error: El número debe ser mayor o igual a 1."

Sino

```
        Para i <- 1 Hasta n Hacer
            Para j <- 1 Hasta i Hacer
                Escribir Sin Saltar "*"
            FinPara
            Escribir ""
        FinPara
    FinSi
FinAlgoritmo
```

CODEBLOCKS

```
#include <stdio.h>

int main() {
    int n, i, j;

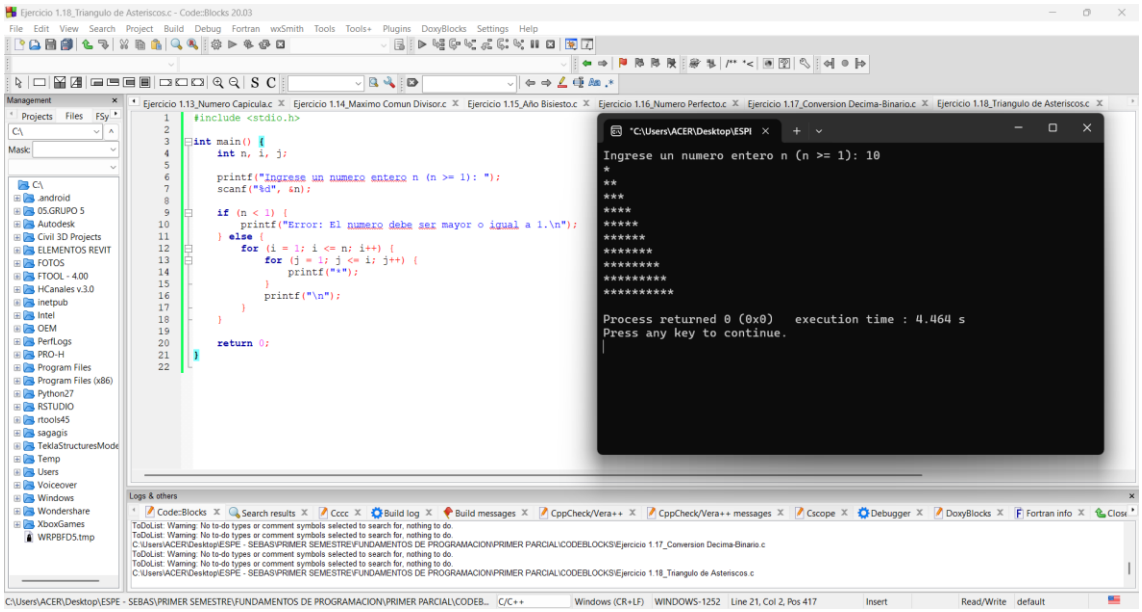
    printf("Ingrese un numero entero n (n >= 1): ");
    scanf("%d", &n);

    if (n < 1) {
        printf("Error: El numero debe ser mayor o igual a 1.\n");
    } else {
        for (i = 1; i <= n; i++) {
            for (j = 1; j <= i; j++) {
                printf("*");
            }
```

```
            printf("\n");
        }
    }

    return 0;
}
```

CAPTURA DE EJECUCIÓN



PROBLEMA 1.19 SEGUNDO MAYOR DE UNA SECUENCIA

Desarrolla un programa que lea números enteros positivos hasta que se introduzca un 0, y muestre el mayor y el segundo mayor de los valores ingresados. Si hay menos de dos valores, debe informarlo. Ejemplo: 7, 3, 9, 5, 0 → mayor = 9, segundo mayor = 7.

Pista: Mantén dos variables, mayor y segundo, y actualízalas con cuidado cuando llegue un valor nuevo. Para pensar: ¿Qué debería pasar si el valor mayor se repite (por ejemplo, 9, 9, 5)? Decide, justifica y prueba tu decisión.

PSEUDOCODIGO

```
Algoritmo SegundoMayor

    Definir num, mayor, segundo, contador Como Entero
```

```
mayor <- 0
segundo <- 0
contador <- 0

Escribir "Ingrese un número entero positivo (0 para terminar):"
Leer num

Mientras num <> 0 Hacer
   contador <- contador + 1

   Si num > mayor Entonces
      segundo <- mayor
      mayor <- num
   Sino
      Si num >= segundo Entonces
         segundo <- num
      FinSi
   FinSi

   Escribir "Ingrese otro número entero positivo (0 para terminar):"
   Leer num
FinMientras
```

```
    Si contador < 2 Entonces
        Escribir "Error: Debe ingresar al menos dos números."
    Sino
        Escribir "El mayor es: ", mayor
        Escribir "El segundo mayor es: ", segundo
    FinSi
FinAlgoritmo
```

CODEBLOCKS

```
#include <stdio.h>

int main() {
    int num, mayor = 0, segundo = 0, contador = 0;

    printf("Ingrese un numero entero positivo (0 para terminar): ");
    scanf("%d", &num);

    while (num != 0) {
        contador++;

        if (num > mayor) {
            segundo = mayor;
            mayor = num;
        } else if (num >= segundo) {
            segundo = num;
```

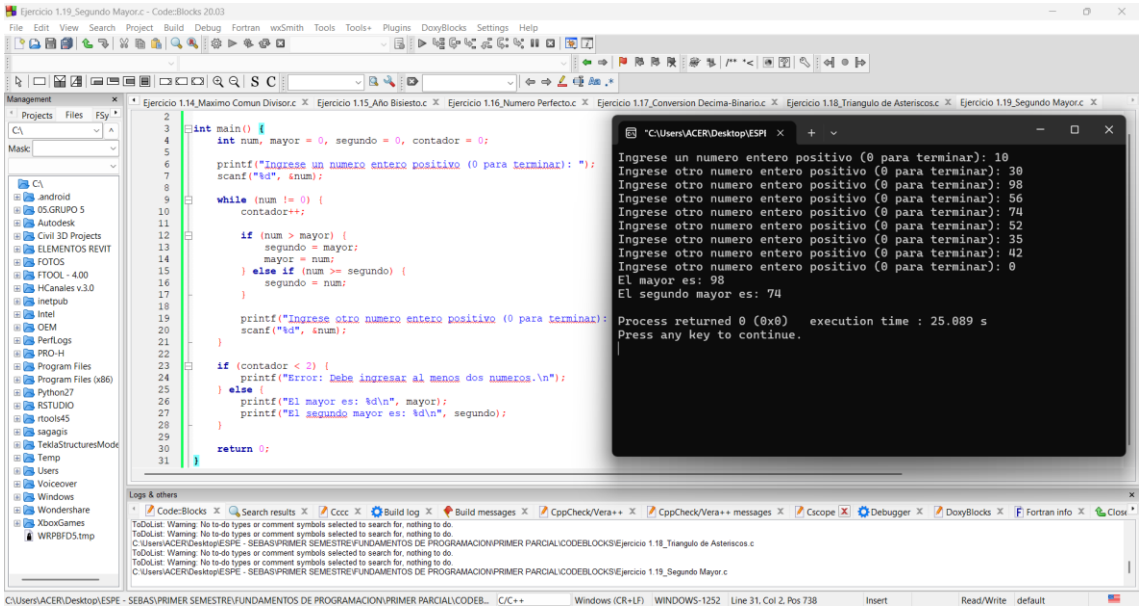
```
        }

        printf("Ingrese otro numero entero positivo (0 para terminar): ");
        scanf("%d", &num);
    }

    if (contador < 2) {
        printf("Error: Debe ingresar al menos dos numeros.\n");
    } else {
        printf("El mayor es: %d\n", mayor);
        printf("El segundo mayor es: %d\n", segundo);
    }

    return 0;
}
```

CAPTURA DE EJECUCIÓN



PROBLEMA 1.20. ADIVINA EL NÚMERO

Desarrolla un programa que “piense” un número entre 1 y 100 (puede ser fijo para la práctica) y pida al usuario adivinarlo. Después de cada intento debe indicar “más alto”, “más bajo” o “¡acertaste!”, y al final mostrar cuántos intentos se usaron.

Pista: Usa un ciclo que se repita hasta que el intento sea igual al número secreto, y un contador de intentos.

Para pensar: Si jugaras con la mejor estrategia posible (dividir siempre el rango por la mitad), ¿cuál es el máximo de intentos que necesitarías para el rango 1–100? Justifica tu respuesta.

PSEUDOCODIGO

```
Algoritmo AdivinaElNumero
    Definir secreto, intento, intentos Como Entero

    secreto <- 42
    intentos <- 0
    intento <- 0

    Mientras intento <> secreto Hacer
        Escribir "Ingrese su intento:"
        Leer intento
        intentos <- intentos + 1

        Si intento < secreto Entonces
            Escribir "Más alto"
        Sino
            Si intento > secreto Entonces
```

```
            Escribir "Más bajo"
        Sino
            Escribir "¡Acertaste!"
        FinSi
    FinSi
FinMientras

Escribir "Adivinaste el número en ", intentos, " intentos."
FinAlgoritmo
```

CODEBLOCKS

```
#include <stdio.h>

int main() {
    int secreto = 42, intento = 0, intentos = 0;

    printf("=== Juego: Adivina el Numero (1 - 100) ===\n");

    while (intento != secreto) {
        printf("Ingrese su intento: ");
        scanf("%d", &intento);
        intentos++;

        if (intento < secreto) {
            printf("Mas alto\n");
```

```
        } else if (intento > secreto) {
            printf("Mas bajo\n");
        } else {
            printf("¡Acertaste!\n");
        }
    }

    printf("Adivinaste el numero en %d intentos.\n", intentos);
    return 0;
}
```

CAPTURA DE EJECUCIÓN

